

Competitive Programming

Section 56 Editorial: Hard Greedy And Matroids Intro

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Read before code: problem dossiers	3
1.2	Problem-specific proof map	3
1.3	Detailed solution guide	3
2	A. Deadline Profit	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
3	B. Minimum Interval Cover	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	7
4	C. MST Savings	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	9
5	D. Watch Schedule	11
5.1	Restatement and greedy choice	11
5.2	Correctness proof	11
5.3	Complexity and implementation notes	11
5.4	Full C++ solution	11
5.5	Full Python solution	12
5.6	Tests that matter	12
6	What to carry forward	13

1 Editorial overview

1.1 Read before code: problem dossiers

A. Deadline Profit. Sort unit jobs by deadline and retain selected profits in a min-heap. After a deadline d , more than d jobs cannot fit; deleting the least profitable keeps the best feasible set of that size. Test equal deadlines and a high-profit late job. Time $O(n \log n)$.

B. Minimum Interval Cover. At the leftmost uncovered point, choose among available intervals the one reaching farthest right. Exchange any optimal cover's first interval for this choice without reducing reach. Test a gap, touching endpoints, and an interval that starts too late. Time $O(n \log n)$.

C. MST Savings. Sum all edges, then use Kruskal/DSU to retain only component-merging edges. The cut property proves their weight is minimum; the difference is removable savings. Test a disconnected graph and parallel edges.

1.2 Problem-specific proof map

A. Deadline Profit. After jobs are sorted by deadline, keep chosen profits in a min-heap. If more jobs than available slots are kept, deleting the least profitable one preserves the best total profit among feasible choices.

B. Minimum Interval Cover. At the current uncovered point, an optimal cover can be exchanged to use an available interval reaching at least as far as the greedy choice. Repeating this reaches the target with the fewest intervals.

C. MST Savings. Kruskal keeps exactly edges that join different components. The cut property proves those edges form a minimum spanning tree; total weight minus its weight is the maximum removable saving.

1.3 Detailed solution guide

A. Scheduling with Deadlines. Sort jobs by deadline and keep the selected profits in a min-heap. After considering all jobs with deadline at most d , the heap contains the best set of jobs that can fit into d slots. If adding a job creates too many selected jobs, remove the smallest profit. This exchange is safe because any feasible schedule with the low-profit job can replace it by the higher-profit job without using more time.

B. Interval Cover. To cover a target segment, repeatedly among all intervals starting at or before the current uncovered point choose the one with farthest right endpoint. Any valid solution must choose some interval covering the current point; choosing the farthest-reaching such interval cannot make the remaining suffix harder to cover.

C. Kruskal Savings. Sort edges by weight and add an edge only if it connects two different components. The DSU maintains the connected components of the partial forest. The cut property proves each chosen edge is safe: it is the lightest available edge crossing some cut. Savings are total edge weight minus the MST weight.

Common pitfalls. Greedy proofs need an exchange or cut argument, not just intuition. For deadline scheduling, deadlines are capacities. For interval cover, detect gaps when no candidate reaches the current point.

The three greedy solutions use exchange arguments.

- A keeps the highest-profit feasible set of unit jobs by deadline using a min-heap.
- B repeatedly chooses, among intervals that already reach the uncovered point, the one extending farthest right.
- C is Kruskal's algorithm: add an edge exactly when it connects two DSU components, then compute removable weight.

Each proof compares the greedy choice with an optimal solution and shows the greedy choice can be exchanged in without worsening feasibility or objective.

2 A. Deadline Profit

2.1 Restatement

Each unit job has deadline and profit. Maximize profit.

2.2 Observations and algorithm

Sort jobs by deadline. After reading every job with deadline at most d , keep the selected profits in a min-heap. If its size exceeds d , delete its least profitable job: no schedule can complete more than d unit jobs by that time. For $(1,5), (1,2), (2,4)$, discard profit 2 and keep 5,4.

```
sort by deadline; push each profit; if heap.size > deadline: pop minimum
```

2.3 Correctness proof

The heap has maximum total profit among feasible selected sets of its size. A violating set must discard one job; removing the minimum profit loses least value, preserving optimality by induction. $\square$

2.4 Complexity

Sorting and heap operations take $O(n \log n)$ time and $O(n)$ memory. In Python use `heapq`; test equal deadlines and a job with deadline zero if allowed.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>
```

2.6 Full Python solution

```
import sys,heapq
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n=data[0]; jobs=sorted((data[i],data[i+1]) for i in range(1,2*n+1,2)); h=[]
    for d,p in jobs:
        heapq.heappush(h,p)
        if len(h)>d: heapq.heappop(h)
    print(sum(h))
if __name__=="__main__": main()
```

3 B. Minimum Interval Cover

3.1 Restatement

Minimum intervals needed to cover $[0, T]$, or -1 .

3.2 Observations and algorithm

Sort intervals by left endpoint. At current covered point `cur`, scan every interval beginning no later than `cur` and choose the greatest right endpoint. If none extends beyond `cur`, coverage is impossible. Trace $[0, 2], [1, 5], [4, 6]$ to choose reach 2, then 5, then 6.

```
while cur < T: best = max right among starts <= cur; fail if best == cur; cur = best
```

3.3 Correctness proof

Every valid cover must choose an interval reaching the current uncovered point. Replacing its first such interval by the farthest-reaching greedy interval cannot reduce coverage of the remaining suffix, proving an exchange step. $\square$

3.4 Complexity

The sort dominates: $O(n \log n)$ time, $O(n)$ memory. Test a gap, touching endpoints, and one interval already covering the whole target.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int T,n;if(!(cin>>T>>n);if(!ci
```

3.6 Full Python solution

```

import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    T,n=data[0],data[1]; seg=sorted((data[i],data[i+1]) for i in range(2,2+2*n,2)); i=ans=0
    while cur<T:
        best=cur
        while i<n and seg[i][0]<=cur: best=max(best,seg[i][1]); i+=1
        if best==cur: print(-1); return
        cur=best; ans+=1
    print(ans)
if __name__=="__main__": main()

```

4 C. MST Savings

4.1 Restatement

Print total edge weight removable while keeping the graph connected, or -1 .

4.2 Observations and algorithm

Sum all edges, then process edges by weight with DSU. Add an edge only when its endpoints are in different components; otherwise it would form a cycle. The kept weight is the MST, so total minus kept is removable weight.

4.3 Correctness proof

By the cut property, each lightest component-joining edge is safe for some MST. After $n-1$ accepted edges the forest is connected and minimum; fewer means the input graph was disconnected. $\square$

4.4 Complexity

Kruskal costs $O(m \log m)$ time and $O(n)$ DSU memory. Use `long long` totals; test parallel edges, equal weights, and a disconnected graph.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(ci
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; edges=[]; total=0; idx=2
    for _ in range(m):
```

```

        a,b,w=data[idx],data[idx+1],data[idx+2]; idx+=3; edges.append((w,a-1,b-1)); total+=w
p=list(range(n))
def f(x):
    while p[x]!=x: p[x]=p[p[x]]; x=p[x]
    return x
keep=0; cnt=0
for w,a,b in sorted(edges):
    ra,rb=f(a),f(b)
    if ra!=rb: p[ra]=rb; keep+=w; cnt+=1
print(total-keep if cnt==n-1 else -1)
if __name__=="__main__": main()

```

5 D. Watch Schedule

5.1 Restatement and greedy choice

Choose the largest number of pairwise non-overlapping half-open intervals. Touching endpoints are compatible, so after choosing $[a, b)$, an interval $[c, d)$ is legal exactly when $c \geq b$.

Sort all intervals by increasing finishing time. Scan them in that order and accept an interval precisely when its start is at least the finish time of the last accepted interval.

For the sample, the sorted scan accepts $[1, 2)$, then $[2, 5)$, then $[5, 7)$. The earlier-ending interval $[0, 4)$ is skipped because it overlaps the chosen first interval.

5.2 Correctness proof

Lemma. After the greedy algorithm chooses its next interval g , there is an optimal schedule whose next interval is g .

Proof. Consider an optimal schedule compatible with the intervals already chosen by greedy, and let its next interval be o . Greedy chose g with the smallest available end time, so $\text{end}(g) \leq \text{end}(o)$. Replace o by g : every later interval of the optimal schedule starts at least $\text{end}(o)$, hence also at least $\text{end}(g)$. The replacement remains feasible and has the same size.

Theorem. The algorithm returns a maximum-size compatible schedule.

Proof. The lemma lets an optimum agree with every greedy choice in turn. When greedy can no longer accept an interval, neither can an optimum that agrees with its prefix. Therefore greedy has optimal size.

5.3 Complexity and implementation notes

Sorting costs $O(n \log n)$ and the scan is $O(n)$, with $O(n)$ memory for the interval list. Sort by end time, not start time or shortest duration. Use `start >= available_from`, not `>`, because the intervals are half-open.

5.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;if(!(cin>>n))return 0;vector<pair<int,int>>
```

5.5 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    intervals = sorted(((data[index], data[index + 1]) for index in range(1, len(data), 2)),
                       key=lambda x: x[1])
    answer = 0
    available_from = -1
    for start, end in intervals:
        if start >= available_from:
            answer += 1
            available_from = end
    print(answer)

if __name__ == "__main__":
    main()
```

5.6 Tests that matter

Intervals that touch exactly at endpoints.
Nested intervals, where the shortest one is not necessarily first in input.
Several equal end times.
One interval covering many mutually compatible shorter intervals.
200000 already compatible intervals.

The tiny oracle enumerates subsets and checks their sorted compatibility. The scale case contains 200000 mutually compatible intervals.

6 What to carry forward